

Презентация к лабораторной работе №5

Основы информационной безопасности

Четвергова Мария Викторовна

20 марта 2025 г.

Российский университет дружбы народов, Москва, Россия

Информация

- Четвергова Мария Викторовна
- студент НПИбд-02-23
- Российский университет дружбы народов
- 1132232886@pfur.ru



Вводная часть

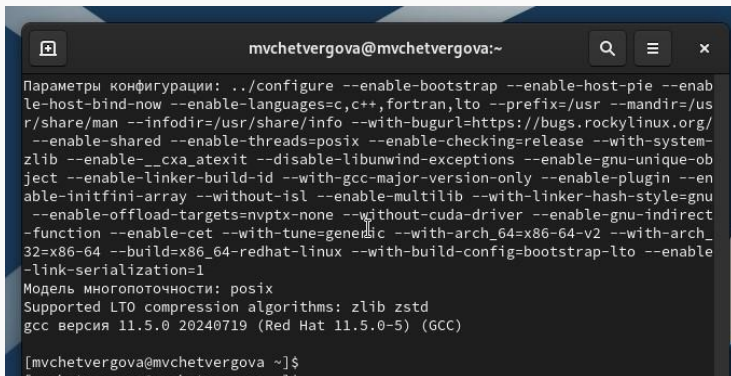
Цель работы

Изучение механизмов изменения идентификаторов, применения SetUID- и Sticky-битов. Получение практических навыков работы в консоли с дополнительными атрибутами. Рассмотрение работы механизма смены идентификатора процессов пользователей, а также влияние бита Sticky на запись и удаление файлов.

Последовательность выполнения работы

Часть 1. Подготовительная работа

Помимо прав администратора для выполнения части заданий потребуются средства разработки приложений. В частности, при подготовке стенда следует убедиться, что в системе установлен компилятор gcc (для этого, например, можно ввести команду `gcc -v`). Если же gcc не установлен, то его необходимо установить, например, командой `yum install gcc`



```
mvchetvergova@mvchetvergova:~  
Параметры конфигурации: ./configure --enable-bootstrap --enable-host-pie --enable-host-bind-now --enable-languages=c,c++,fortran,lto --prefix=/usr --mandir=/usr/share/man --infodir=/usr/share/info --with-bugurl=https://bugs.rockylinux.org/ --enable-shared --enable-threads=posix --enable-checking=release --with-system-zlib --enable-__cxa_atexit --disable-libunwind-exceptions --enable-gnu-unique-object --enable-linker-build-id --with-gcc-major-version-only --enable-plugin --enable-initfini-array --without-isl --enable-multilib --with-linker-hash-style=gnu --enable-offload-targets=nvptx-none --without-cuda-driver --enable-gnu-indirect-function --enable-cet --with-tune=generic --with-arch_64=x86_64-v2 --with-arch_32=x86_64 --build=x86_64-redhat-linux --with-build-config=bootstrap-lto --enable-link-serialization=1  
Модель многопоточности: posix  
Supported LTO compression algorithms: zlib zstd  
gcc версия 11.5.0 20240719 (Red Hat 11.5.0-5) (GCC)  
[mvchetvergova@mvchetvergova ~]$
```

Часть 1. Подготовительная работа

```
[mvchetvergova@mvchetvergova ~]$  
[mvchetvergova@mvchetvergova ~]$  
[mvchetvergova@mvchetvergova ~]$ setenforce 0  
setenforce: security_setenforce() failed: Permission denied  
[mvchetvergova@mvchetvergova ~]$
```


Часть 1. Подготовительная работа

Компилятор языка C называется gcc. Компилятор языка C++ называется g++ и запускается с параметрами почти так же, как gcc. Проверить это можно следующими командами: `whereis gcc` `whereis g++`

```
[mvchetvergova@mvchetvergova ~]$  
[mvchetvergova@mvchetvergova ~]$  
[mvchetvergova@mvchetvergova ~]$  
[mvchetvergova@mvchetvergova ~]$ whereis gcc  
gcc: /usr/bin/gcc /usr/lib/gcc /usr/libexec/gcc /usr/share/man/man1/gcc.1.gz /usr/share/info/gcc.info.gz  
[mvchetvergova@mvchetvergova ~]$  
[mvchetvergova@mvchetvergova ~]$ whereis g++  
g++:  
[mvchetvergova@mvchetvergova ~]$  
[mvchetvergova@mvchetvergova ~]$
```

Часть 1. Подготовительная работа

Логично спросить: если установление SetUID- и SetGID- битов на сценарий не приводит к нужному результату как с исполняемыми файлами, то что мешает установить эти биты на сам интерпретатор? Ничего не мешает, только их установление приведёт к тому, что, так как владельцем /bin/bash является root: `ls -l /bin/bash`

```
g++:  
[mvchetvergova@mvchetvergova ~]$  
[mvchetvergova@mvchetvergova ~]$ ls -l /bin/bash  
-rwxr-xr-x. 1 root root 1389024 anp 30 2024 /bin/bash  
[mvchetvergova@mvchetvergova ~]$  
[mvchetvergova@mvchetvergova ~]$
```

Часть 2. Создание программы

1. Войдите в систему от имени пользователя guest.

```
[guest@mvchetvergova ~]$ ls
dir1  Видео      Загрузки      Музыка      'Рабочий стол'
dir2  Документы  Изображения  Общедоступные  Шаблоны
[guest@mvchetvergova ~]$ mkdir lab5
[guest@mvchetvergova ~]$ ls
dir1  lab5  Документы  Изображения  Общедоступные  Шаблоны
dir2  Видео  Загрузки  Музыка      'Рабочий стол'
[guest@mvchetvergova ~]$ cd lab5
[guest@mvchetvergova lab5]$ ls
[guest@mvchetvergova lab5]$ touch simpleid.c
[guest@mvchetvergova lab5]$ ls
simpleid.c
[guest@mvchetvergova lab5]$ gedit
[guest@mvchetvergova lab5]$ gedit simpled.c
```

Часть 2. Создание программы

2. Создайте программу simpleid.c:



The screenshot shows a code editor window with a dark theme. The title bar at the top contains three icons: a green circle with a white 'O', a document icon, and the text 'Текстовый редактор'. To the right of the title bar is the date and time 'Чт, 17 апреля 13:15'. Below the title bar is a toolbar with a button labeled 'Открыть' with a dropdown arrow, a button with a plus icon in a square, and a button labeled 'Сохранить'. The main area of the window displays the code for 'simpled.c' located at '~/lab5'. The code is as follows:

```
1 #include <sys/types.h>
2 #include <unistd.h>
3 #include <stdio.h>
4 int
5 main ()
6 {
7     uid_t uid = geteuid ();
8     gid_t gid = getegid ();
9     printf ("uid=%d, gid=%d\n", uid, gid);
10    return 0;
11 }
```

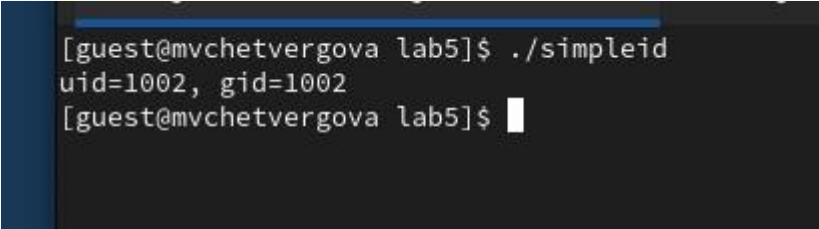
Часть 2. Создание программы

3. Скомпилируйте программу и убедитесь, что файл программы создан:
`gcc simpleid.c -o simpleid`

```
simpleid.c
[guest@mvchetvergova lab5]$ gcc simpleid.c -o simpleid
[guest@mvchetvergova lab5]$ ls
simpleid  simpleid.c
```

```
[guest@mvchetvergova lab5]$ gcc simpleid.c -o simpleid
[guest@mvchetvergova lab5]$ ls
simpleid  simpleid.c
[guest@mvchetvergova lab5]$ gcc -c simpleid.c
[guest@mvchetvergova lab5]$ ls
simpleid  simpleid.c  simpleid.o
[guest@mvchetvergova lab5]$
```

4. Выполните программу simpleid:
./simpleid

A terminal window with a dark background and a blue title bar. The prompt is [guest@mvchetvergova lab5]\$. The command ./simpleid has been entered and executed, resulting in the output uid=1002, gid=1002. The prompt is now [guest@mvchetvergova lab5]\$ with a white cursor.

```
[guest@mvchetvergova lab5]$ ./simpleid
uid=1002, gid=1002
[guest@mvchetvergova lab5]$
```

Часть 2. Создание программы

5. Выполните системную программу `id`:

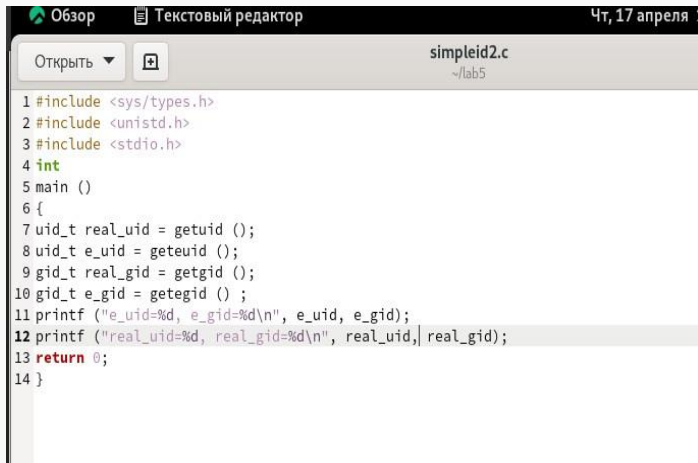
`id`

и сравните полученный вами результат с данными предыдущего пункта задания.

```
[guest@mvchetvergova lab5]$ ./simpleid
uid=1002, gid=1002
[guest@mvchetvergova lab5]$
[guest@mvchetvergova lab5]$
[guest@mvchetvergova lab5]$ id
uid=1002(guest) gid=1002(guest) группы=1002(guest) контекст=unconfined_u:unconfi
ned_r:unconfined_t:s0-s0:c0.c1023
[guest@mvchetvergova lab5]$
```

Часть 2. Создание программы

6. Усложните программу, добавив вывод действительных идентификаторов:



```
Обзор | Текстовый редактор | Чт, 17 апреля 1
simpleid2.c
~/lab5

1 #include <sys/types.h>
2 #include <unistd.h>
3 #include <stdio.h>
4 int
5 main ()
6 {
7     uid_t real_uid = getuid ();
8     uid_t e_uid = geteuid ();
9     gid_t real_gid = getgid ();
10    gid_t e_gid = getegid ();
11    printf ("e_uid=%d, e_gid=%d\n", e_uid, e_gid);
12    printf ("real_uid=%d, real_gid=%d\n", real_uid, real_gid);
13    return 0;
14 }
```


Часть 2. Создание программы

7. Скомпилируйте и запустите simpleid2.c:

```
gcc simpleid2.c -o simpleid2
```

```
./simpleid2
```

```
[guest@mvchettergova lab5]$ gedit simpleid2.c
[guest@mvchettergova lab5]$ gcc simpleid2.c -o simpleid2
[guest@mvchettergova lab5]$ ls
simpleid  simpleid2  simpleid2.c  simpleid.c
[guest@mvchettergova lab5]$ gedit simpleid2.c
[guest@mvchettergova lab5]$ ./simpleid2
e_uid=1002, e_gid=1002
real_uid=1002, real_gid=1002
[guest@mvchettergova lab5]$
```

Часть 2. Создание программы

8. От имени суперпользователя выполните команды:

Информационная безопасность компьютерных сетей 37

```
chown root:guest /home/guest/simpleid2
```

```
chmod u+s /home/guest/simpleid2
```

```
-rwxr-xr-x. 1 guest guest 17656 anp 17 13:32 simpleid2
[root@mvchetvergova lab5]#
[root@mvchetvergova lab5]#
[root@mvchetvergova lab5]# chown root:guest /home/guest/lab5/simpleid2
[root@mvchetvergova lab5]# chmod u+s /home/guest/lab5/simpleid2
[root@mvchetvergova lab5]#
[root@mvchetvergova lab5]#
```

Часть 2. Создание программы

9. Используйте `sudo` или повысьте временно свои права с помощью `su`. Поясните, что делают эти команды.

```
[root@mvchetvergova lab5]#  
[root@mvchetvergova lab5]#  
[root@mvchetvergova lab5]# ls -l simpleid2  
-rwsr-xr-x. 1 root guest 17656 anp 17 13:32 simpleid2  
[root@mvchetvergova lab5]#
```

Часть 2. Создание программы

10. Выполните проверку правильности установки новых атрибутов и смены владельца файла simpleid2:

`ls -l simpleid2`

```
[root@mvchetvergova lab5]#  
[root@mvchetvergova lab5]#  
[root@mvchetvergova lab5]# ls -l simpleid2  
-rwsr-xr-x. 1 root guest 17656 anp 17 13:32 simpleid2  
[root@mvchetvergova lab5]#
```

Часть 2. Создание программы

11. Запустите simpleid2 и id:

./simpleid2

id

Сравните результаты.

```
[root@mvchetvergova lab5]#  
[root@mvchetvergova lab5]# ./simpleid2  
e_uid=0, e_gid=0  
real_uid=0, real_gid=0  
[root@mvchetvergova lab5]# id  
uid=0(root) gid=0(root) группы=0(root) контекст=unconfined_u:unconfined_r:unconf  
ined_t:s0-s0:c0.c1023  
[root@mvchetvergova lab5]#
```

Часть 2. Создание программы

13. Создайте программу readfile.c:

```
Открыть ▼  readfile.c  
/home/guest/lab5  
1 #include <fcntl.h>  
2 #include <stdio.h>  
3 #include <sys/stat.h>  
4 #include <sys/types.h>  
5 #include <unistd.h>  
6 int  
7 main (int argc, char* argv[])  
8 {  
9     unsigned char buffer[16];  
10    size_t bytes_read;  
11    int i;  
12    int fd = open (argv[1], O_RDONLY);  
13    do  
14    {  
15        bytes_read = read (fd, buffer, sizeof (buffer));  
16        for (i = 0; i < bytes_read; ++i) printf("%c", buffer[i]);  
17    }  
18    while (bytes_read == sizeof (buffer));  
19    close (fd);  
20    return 0;  
21 }
```

Часть 2. Создание программы

14. Откомпилируйте её.
gcc readfile.c -o readfile

```
[root@mvchetvergova lab5]#  
[root@mvchetvergova lab5]#  
[root@mvchetvergova lab5]# gcc readfile.c -o readfile  
[root@mvchetvergova lab5]# ls  
readfile readfile.c simpleid simpleid2 simpleid2.c simpleid.c  
[root@mvchetvergova lab5]#  
[root@mvchetvergova lab5]#
```

Часть 3. Исследование Sticky-бита

1. Выясните, установлен ли атрибут Sticky на директории /tmp, для чего выполните команду

`ls -l / | grep tmp`

```
[guest@mvchetvergova ~]$ ls -l / | grep tmp
drwxrwxrwt. 17 root root 4096 апр 17 13:47 tmp
[guest@mvchetvergova ~]$
```


Часть 3. Исследование Sticky-бита

2. От имени пользователя guest создайте файл file01.txt в директории /tmp со словом test:

```
echo "test" > /tmp/file01.txt
```

A terminal window with a dark background. The prompt is [guest@mvchetvergova ~]\$. The command echo "test" > /tmp/file01.txt is entered and executed. The prompt returns to [guest@mvchetvergova ~]\$.

```
[guest@mvchetvergova ~]$ echo "test" > /tmp/file01.txt  
[guest@mvchetvergova ~]$
```

Часть 3. Исследование Sticky-бита

3. Просмотрите атрибуты у только что созданного файла и разрешите чтение и запись для категории пользователей «все остальные»:

```
ls -l /tmp/file01.txt
```

```
chmod o+rw /tmp/file01.txt
```

```
ls -l /tmp/file01.txt
```

```
[guest@mvchetvergova ~]$  
[guest@mvchetvergova ~]$  
[guest@mvchetvergova ~]$ ls -l /tmp/file01.txt  
-rw-r--r--. 1 guest guest 5 anp 17 13:49 /tmp/file01.txt  
[guest@mvchetvergova ~]$  
[guest@mvchetvergova ~]$ chmod o+rw /tmp/file01.txt  
[guest@mvchetvergova ~]$  
[guest@mvchetvergova ~]$ ls -l /tmp/file01.txt  
-rw-r--rw-. 1 guest guest 5 anp 17 13:49 /tmp/file01.txt  
[guest@mvchetvergova ~]$  
[guest@mvchetvergova ~]$
```

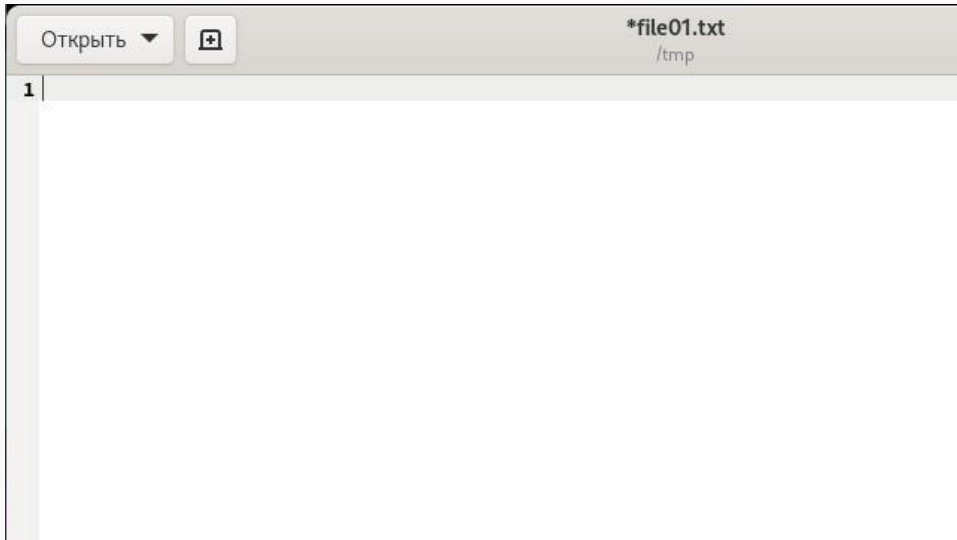
Часть 3. Исследование Sticky-бита

4. От пользователя guest2 (не являющегося владельцем) попробуйте прочитать файл /tmp/file01.txt:

cat /tmp/file01.txt

```
[guest@mvchetvergova ~]$  
[guest@mvchetvergova ~]$ echo "test3" > /tmp/file01.txt  
[guest@mvchetvergova ~]$ gedit /tmp/file01.txt  
[guest@mvchetvergova ~]$
```

Часть 3. Исследование Sticky-бита



Часть 3. Исследование Sticky-бита

5. От пользователя guest2 попробуйте дозаписать в файл /tmp/file01.txt слово test2 командой

```
echo "test2" > /tmp/file01.txt
```

Удалось ли вам выполнить операцию? – не удалось

```
[guest@mvchetvergova ~]$  
[guest@mvchetvergova ~]$  
[guest@mvchetvergova ~]$ cat /tmp/file01.txt  
[guest@mvchetvergova ~]$
```

Часть 3. Исследование Sticky-бита

6. Проверьте содержимое файла командой
`cat /tmp/file01.txt`

```
[guest@mvchetvergova ~]$  
[guest@mvchetvergova ~]$ cat /tmp/file01.txt  
[guest@mvchetvergova ~]$ echo "test3" > /tmp/file01.txt  
[guest@mvchetvergova ~]$ cat /tmp/file01.txt  
test3  
[guest@mvchetvergova ~]$
```

Часть 3. Исследование Sticky-бита

7. От пользователя guest2 попробуйте записать в файл /tmp/file01.txt слово test3, стерев при этом всю имеющуюся в файле информацию командой `echo "test3" > /tmp/file01.txt`

```
[guest@mvchetvergova ~]$ ls -l / | grep tmp
drwxrwxrwt. 17 root root 4096 апр 17 13:47 tmp
[guest@mvchetvergova ~]$
```

В ходе выполнения Лабораторной работы №5 были изучены механизмы изменения идентификаторов, применения SetUID- и Sticky-битов. Получены практических навыков работы в консоли с дополнительными атрибутами. Рассмотрены работы механизма смены идентификатора процессов пользователей, а также влияние бита Sticky на запись и удаление файлов.

Список литературы

Список литературы

1. Поттеринг Л. Systemd для администраторов: цикл статей. — 2010. — URL: <http://wiki.opennet.ru/Systemd>.
2. Neil N. J. Learning CentOS: A Beginners Guide to Learning Linux. — CreateSpace Independent Publishing Platform, 2016.
3. Systemd. — 2022. — URL: <https://wiki.archlinux.org/title/Systemd>.

Спасибо за внимание